Incremental View Maintenance as an Embeddable Library

Technical notes behind the idea.

What this note covers

These are the deeper notes behind a short idea: the evidence that the gap is real, the prior art and where each piece stops, and a possible design. It deliberately skips the high-level framing and goes to the substance.

The missing middle

There are two regimes today for keeping a derived view (a dashboard, an aggregate, a computed table) current as its inputs change:

- **Recompute on a schedule.** Easy to build, but it rescans everything even when one row moved, and the view always lags the data.
- **Run a streaming dataflow system** (Flink with ksqlDB, Materialize, RisingWave). Correct and incremental, but each is a separate distributed service to deploy, fund, and operate.

Between these sits an empty slot: an in-process component, linked into the application the way SQLite or DuckDB is, that keeps a declared SQL view current by consuming only the changes to its inputs, with no external server. No production-quality library occupies that slot today.

Evidence the gap is current

The people closest to the problem keep hitting it:

- **PostgreSQL**: incremental maintenance lives only in the out-of-core pg_ivm extension; core has no merged support, so REFRESH MATERIALIZED VIEW still recomputes the whole query.
- **SQLite**: no native support at all. The usual workaround is hand-written trigger chains, which do not compose across joins and aggregations.
- A long *Ask HN* thread, “Incremental View Maintenance for SQLite?” (2023), drew over a hundred comments and reached no satisfying answer.
- The cr-sqlite maintainer (Discussion #309) lists full incremental maintenance as a known missing piece and documents why each current approach falls short.

These are independent signals from people building real systems, not a single complaint.

Why the timing is good

The reason the slot is still empty looks like timing: the theory to do this correctly for rich SQL only matured recently.

- **DBSP**, published at VLDB 2023 (best paper), gives a clean algebraic foundation (a “Z-set” / delta algebra) and derives the incremental version of a query mechanically.
- **OpenIVM** (SIGMOD 2024) demonstrated a working SQL-to-SQL incremental compiler on top of DuckDB.
- Earlier attempts aged out: **DBToaster** generates C++ and has been dormant since around 2017; **differential-dataflow** is powerful but exposes no SQL surface and has a steep learning curve.

So the math needed to build this correctly is roughly two years old, and a clean, embeddable artifact built on it does not yet exist.

Existing options and where each stops

Tool	Form	Why it does not fill the slot
Materialize / RisingWave	Managed or standalone streaming DB	Separate service; real minimum footprint and cost
Feldera (DBSP)	Containerized runtime	Infrastructure, not an embeddable library
DBToaster	C++ code generator	Dormant since ~2017; impractical to embed in modern stacks
differential-dataflow	Rust dataflow library	No SQL interface; steep; needs timely-dataflow expertise
cr-sqlite	SQLite extension (CRDT + partial IVM)	Must mark tables at creation; cannot retrofit; work in progress
Manual SQL triggers	Hand-written	Brittle; do not compose on joins or aggregations
pg_ivm	PostgreSQL extension	Postgres-only, out of core, not embeddable elsewhere

A possible approach

A credible design has three layers:

1. **A delta-algebra core** (Z-sets and the DBSP operators). Given a change to base data, it produces the matching change to each view.
2. **A SQL-to-plan compiler** that lowers an ordinary SELECT (filters, joins, GROUP BY) into a maintainable operator graph instead of re-executing the query.
3. **A change-capture adapter** that feeds deltas in: SQLite's update hook, a PostgreSQL WAL stream, or an in-process mutation log.

Two choices would make it genuinely useful in practice:

- Make the operator graph **serializable and resumable**, so a process restart does not force a full recompute.
- Add **query-aware compaction** to bound memory growth.

A **WebAssembly** build would carry the same engine into the browser, which is the underserved local-first case.

The hard parts (and why this is a real systems and PL problem, not a weekend project): correctness across the full SQL surface (outer joins, aggregations that must handle deletions, recursive queries), bounding state growth, and a credible correctness story (property-based testing against a recompute oracle, or proofs carried by the DBSP algebra). A silent wrong answer after a delete is the failure mode to design against from the start.

Questions worth investigating

- How much of SQL can be incrementalized cleanly before operator state becomes impractical to maintain?
- What is the right correctness guarantee: testing against a recompute oracle, or a formal proof carried by the algebra?
- Is the in-process framing actually viable, or does change capture eventually push the design back toward an external service?

References

- *DBSP: Automatic Incremental View Maintenance for Rich Query Languages.* VLDB 2023.
- *OpenIVM: a SQL-to-SQL compiler for incremental computation on DuckDB.* SIGMOD 2024.
- Ask HN: Incremental View Maintenance for SQLite? (2023)

- cr-sqlite, Discussion #309: reactivity and live queries
- pg_ivm: incremental view maintenance extension for PostgreSQL.
- Feldera: a commercial engine built on the DBSP framework.